WISEConv: Closing the Sparsity-to-Speedup Gap in Masked Convolution with Worklist-Driven Execution

Anonymous Author(s)

Abstract

Masked convolution promises lower latency by computing only positions an upstream policy marks active, yet existing paths keep either throughput (tile skipping) or useful-compute ratio (gather-scatter), never both, failing to turn sparsity into proportional speedup. Contrary to the expectation that scattered work breaks dense efficiency, we find that restricting a dense tiled pipeline to scattered positions largely preserves per-tile throughput: irregularity is confined to row-level indirection outside the reduction loop, leaving channel-contiguous access and weight-tile reuse identical to dense. We therefore present WISEConv, which drives the unchanged dense pipeline with an active-output worklist. The worklist brings two costs dense execution never pays. Construction cost stays fixed per layer shape as compute shrinks with the active ratio, so WISEConv builds the worklist in one mask-space pass and reuses it across compatible operators. The worklist length, visible only on the device, varies per frame and layer and shifts the best tile configuration, so WISEConv performs on-device cost-aware configuration switching, weighing each candidate's gain against its selection cost. Across six GPUs and four workloads, WISEConv achieves a 2.20× geometric-mean latency reduction over the fastest baseline per workload-platform pair and reduces energy by 60.8–90.1% versus dense execution on embedded GPUs.

Keywords: Sparse convolution, dynamic spatial sparsity, GPU runtime, deep learning systems, efficient inference

1 Introduction

Dense convolution dominates per-frame latency in vision tasks where reaction time decides the outcome [3, 12, 43], such as a car avoiding a pedestrian stepping out of occlusion [25], a drone dodging an obstacle [13], or a robot returning a table-tennis ball [9] (Figure 1). Yet on a given input much of that computation is spatially redundant. A common remedy is masked convolution: an upstream policy marks spatially active positions on the dense feature grid, and a dedicated sparse convolution operator updates only these positions in the dense output. The mask may derive from event-camera increments [10, 27], temporal differences [20, 32], or learned spatial gates [24, 40]. These mask sources promise large reductions in convolution work, yet existing masked-convolution systems [10, 24, 32, 40] rarely translate the sparsity into proportional latency reduction.

Dense GPU convolution obtains high *throughput* (operations per second) from coordinate-driven, tiled execution.

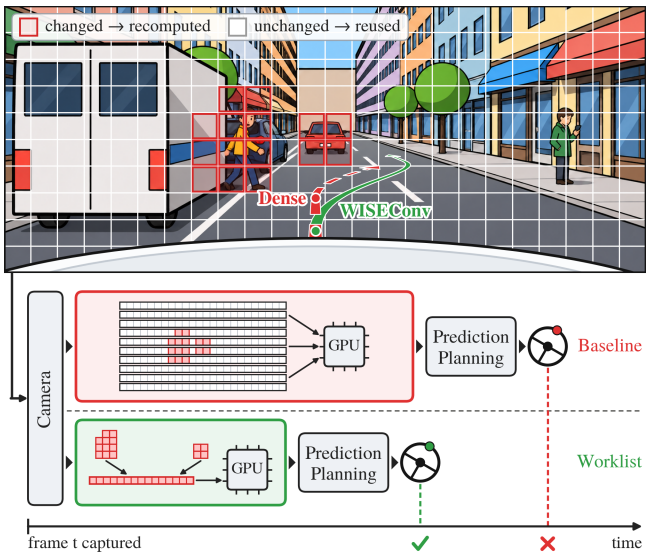


Figure 1. Perception latency decides whether a vehicle can react in time. Masked convolution needs only the changed positions (blue). WISEConv drives the dense tiled pipeline with a worklist of these positions (the sequence of their output coordinates); computing scattered positions this way keeps the throughput of dense execution, so convolution shrinks with the active positions and the vehicle acts earlier.

Each tile enumerates a contiguous block of output coordinates; those coordinates fix each output position's receptive field and write location, while the reduction over channels and kernel offsets follows a regular datapath with coalesced loads and weight-tile reuse. In masked convolution, reducing latency proportionally requires sustaining this throughput while achieving a high *useful-compute ratio* (the fraction of issued operations that produce needed outputs), but the per-frame position selection imposed by the mask conflicts with the contiguous coordinate assumption that sustains throughput, placing these two axes in tension.

Existing systems sacrifice one axis for the other. *Gather-scatter* [24, 40, 44] extracts exactly the active positions into a packed representation, computes, and scatters the results back, but is bottlenecked by the memory traffic of materializing their receptive fields. *Tile skipping* [10, 20, 32, 34] suppresses tiles that are entirely inactive and retains the dense pipeline, but wastes computation on partially active tiles. On an RTX 3080 running YOLOv8m [22] at batch 4 under temporal difference masks [32], 82.2% of convolution work is unnecessary. Gather-scatter sustains only 43.7% of

the dense pipeline's throughput, reducing latency by just 59.3%; tile skipping reaches higher throughput but wastes 32.6% of issued operations, reducing latency by 54.3%, slower than gather-scatter despite its throughput advantage (§2.2).

We find that redirecting a dense tiled pipeline to compute only scattered active positions achieves both a high useful-compute ratio and the throughput of dense execution. We collect these positions into a worklist and characterize (§3) the resulting tiled execution on a desktop and an embedded GPU, identifying why: because the input stays a dense grid, the irregularity reduces to one index per output row, outside the reduction loop that drives coalesced channel loads and weight-tile reuse, so per-tile throughput matches dense execution. When the tile configuration follows the worklist length, convolution time is affine in the active ratio with a slope matching the dense marginal cost; the residual gap is an intercept of 5.6–17.4% of dense time per layer (median per GPU and precision), dominated by kernel launch latency and fixed per-tile stalls such as barriers, not by throughput loss.

While this finding establishes that worklist-driven tiled execution can sustain both axes, closing the gap end to end requires solving two further challenges. First, *worklist construction cost*: deriving the worklist from the mask requires scanning the whole grid and costs time independent of the active count, so construction's share of latency grows as activity falls and accumulates when every layer rebuilds. Second, *worklist consumption efficiency*: the affine property holds for a given number of active positions under a matching tile configuration, but in deployment this number is visible only on the device after construction and shifts across frames and layers. Reading it back to the host stalls the per-frame path; adapting on the device avoids the stall but incurs a non-negligible cost per decision point (§3).

Based on these findings, we present WISEConv (**W**orklist-dr**I**ven ma**S**k**E**d **Conv**olution), which drives the dense tiled pipeline with an active-output worklist to achieve both a high useful-compute ratio and high throughput by co-designing worklist construction and consumption. To reduce construction cost, a single mask-space pass fuses output-mask propagation with per-tile compaction, operating on only mask and coordinate metadata; compatible operators reuse the worklist instead of reconstructing it. To sustain consumption efficiency, a persistent compute grid consumes the worklist without host synchronization, and a cost-aware selection mechanism adapts the tile configuration per frame, adding a candidate to a layer only where its expected gain exceeds the selection overhead.

This paper makes the following contributions:

- We characterize worklist-driven tiled execution, identifying why computing scattered active positions preserves dense-level throughput and where the remaining gap comes from. These findings motivate and constrain the design (§3).
- We present WISEConv, which co-designs worklist construction, worklist-driven tiled compute, and cost-aware tile selection to jointly sustain high useful-compute ratio and high throughput (§4).
- We evaluate WISEConv on four workloads across six GPUs. WISEConv achieves a 2.18× (FP32) and 2.22× (FP16) geometric-mean speedup over the fastest baseline per pair, and reduces energy by 60.8–90.1% relative to dense execution on embedded GPUs (§6).

2 Background and Motivation

2.1 Masked Convolution

Convolution runs inside the perception-action loop of many vision applications [3, 36, 41, 43, 47], where per-frame inference latency bounds system responsiveness [11–13, 21]. To cut this latency, masked convolution exploits dynamic spatial sparsity: an upstream policy emits a spatial mask, and the operator computes only the marked positions. The mask comes from sources including event-camera increments [10, 27], temporal residuals between video frames [4, 20, 31, 32, 42], or learned input-dependent gates [15, 24, 40, 44]. These sources differ in sensing modality and policy, but all expose the same dynamic spatial sparsity on a dense 2D feature grid, so a single masked-convolution runtime serves them all.

Masked convolution retains dense tensor layouts and arithmetic, restricting only which output coordinates are computed (Figure 2). It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$, a weight tensor $\mathbf{f} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ of kernel

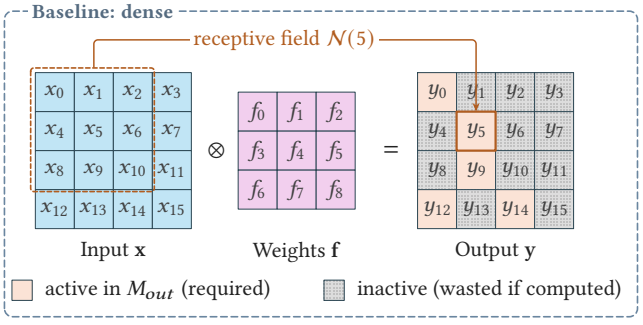


Figure 2. Masked convolution and its dense execution. Each output y_p aggregates its receptive field $\mathcal{N}(p)$ in $\mathbf{x}$ with the weights $\mathbf{f}$. Dense convolution computes every output in tiles of contiguous positions, which keeps loads coalesced and each weight tile reused across the tile, so it runs at high throughput; but only the outputs active in M_{out} (colored) are required, and the throughput spent on the rest (dotted) is wasted.

size k, and an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \ldots, H_{out} W_{out}\}$, where each coordinate $p \in \mathcal{G}$ indexes a spatial position (h, w) and has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0,1\}^{H_{out} \times W_{out}}$. Spatial gating predicts M_{out} directly; event and temporal policies instead supply an input mask $M_{in} \in \{0,1\}^{H_{in} \times W_{in}}$ whose active positions propagate their influence along the receptive field, so p stays active whenever any input in $\mathcal{N}(p)$ is active,

$$M_{out}[p] = \begin{cases} 1, & \exists r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

M_{out} thus identifies the output positions that the execution path is required to compute, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{f}$.

An execution path issues computation over a sequence of position slots C, where each slot is drawn from $\mathcal{G} \cup \{\perp\}$; $\perp$ denotes a padding slot that carries no output coordinate. The path must satisfy the *coverage contract*: every active coordinate is included, $\{p : M_{out}[p] = 1\} \subseteq \{c \in C : c \neq \perp\}$. For each non-$\perp$ slot $p \in C$ it computes

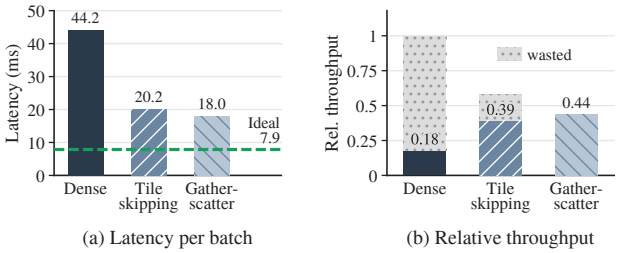
$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{x}[\mathcal{N}_i(p)]\, \mathbf{f}_i, \tag{2}$$

where $\mathcal{N}_i(p)$ is the i-th position in $\mathcal{N}(p)$ and $\mathbf{f}_i \in \mathbb{R}^{C_{in} \times C_{out}}$ the corresponding weight slice. How unrequested positions are treated is policy-specific: event and temporal execution retains prior state, while learned gating merges results into a residual. Coverage is therefore the operator-level correctness condition; the policy determines the treatment of all other positions. Beyond coverage, a path may issue inactive or $\perp$ slots; dense convolution is the extreme where $\{c \in C : c \neq \perp\} = \mathcal{G}$. The policy fixes the accuracy and required work.

2.2 The Sparsity-to-Latency Gap

Existing systems execute a mask along one of two paths. *Tile skipping* [10, 20, 26, 31, 34, 42] (Figure 4) inherits the dense pipeline's tile-based computation without additional metadata passes, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [32] fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, to cut wasted work. *Gather-scatter* [24, 44] (Figure 5) instead abandons the dense pipeline: it extracts exactly the active coordinates into a packed representation, runs a separate GEMM, and scatters the results back, removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [40] adopts a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

A policy fixes the active ratio α, defined as the fraction of output-grid positions marked active, whereas a concrete execution path determines the useful-compute ratio η, defined



(a) Latency per batch (b) Relative throughput

Figure 3. YOLOv8m [22] at batch 4 on WildTrack 640 and RTX 3080 (FP32) under the DeltaCNN [32] temporal difference masks, comparing cuDNN (Dense), DeltaCNN tile skipping, and MMCV gather-scatter [6]; all paths consume the same masks. (a) End-to-end latency per batch step; the dashed line (Ideal) is dense latency scaled by the active ratio of the convolution work. (b) Throughput normalized to dense raw throughput. Bar height is raw throughput T; the colored share is effective throughput T_{eff} (labeled) and the dotted share is spent on inactive positions.

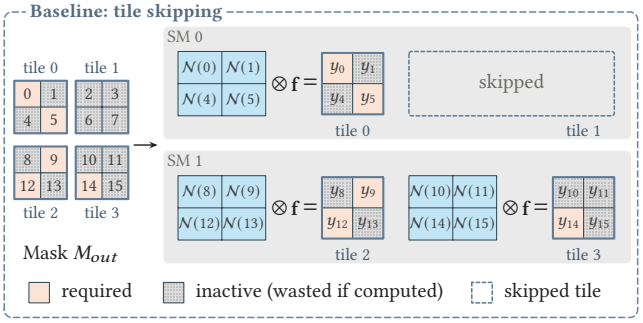


Figure 4. Tile skipping on the example of Figure 2. It keeps the dense tiled pipeline, so each issued tile runs at dense throughput without extra passes, and skips only tiles with no active position. A partially active tile still runs in full, so its inactive (dotted) outputs are wasted computation.

as the fraction of issued slots that correspond to required outputs:

$$\alpha = \frac{\|M_{out}\|_0}{|\mathcal{G}|}, \quad \eta = \frac{\|M_{out}\|_0}{|C|}. \tag{3}$$

For $|C| > 0$, an ideal path achieves $\eta = 1$ by issuing only required outputs; dense convolution, which issues the entire grid, has $\eta \approx \alpha$. An execution path also determines its throughput T, which combines with η to yield the effective throughput T_{eff}:

$$T = \frac{2|C|k^2 C_{in} C_{out}}{t}, \quad T_{\text{eff}} = \eta T. \tag{4}$$

where t denotes the latency of the execution path, including auxiliary mask and coordinate processing, k is the filter kernel size, and C_{in} and C_{out} are the input and output channel

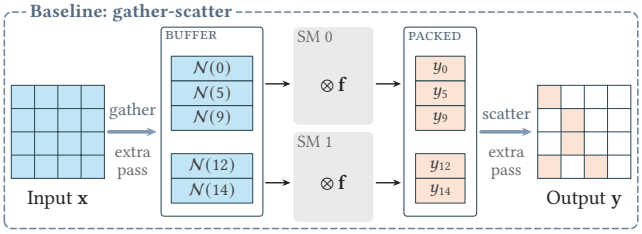


Figure 5. Gather-scatter on the example of Figure 2. To compute only active outputs, it copies their receptive fields into a packed buffer (gather) so that a GEMM can process them as contiguous rows, then writes the packed results back into y (scatter). No computation is wasted, but the two extra passes over memory, not the computation, dominate its latency.

counts respectively. The factor of two counts each multiply-accumulate as two FLOPs. Consequently, T_{eff} directly characterizes the processing rate of required outputs and therefore determines the realized latency.

Figure 3 makes this concrete on RTX 3080 for YOLOv8m [22] at batch 4 under temporal difference masks [32], where 82.2% of convolution work is unnecessary. Tile skipping holds throughput at 57.8% of dense but at a useful-compute ratio of only 67.4%; gather-scatter lifts the useful-compute ratio to one, yet extraction traffic collapses its throughput to 43.7% of dense. Neither path efficiently converts the GPU's capability into effective throughput.

2.3 Distinction from Related Sparse Execution

WISEConv differs from other sparse-execution work in representation and sparsity type. 3D sparse-convolution systems [8, 19, 35, 38, 39, 45] operate on sparse point-cloud coordinates; sparse matrix multiplication [14, 17, 33] targets structural sparsity in graphs or pruned weights; structured weight pruning [28] exploits fixed patterns; sparse-format compilers [23, 46] encode sparsity in tensor formats. DPACS [18] targets the same dynamic 2D spatial sparsity but designs a custom dataflow accelerator to handle the resulting irregular access patterns.

Row-indexed operand loads are an established implementation technique: CUTLASS fuses gather and scatter into GEMM [1], mixture-of-experts kernels read tokens through scattered indices inside a grouped GEMM [16, 37], and the Blackwell TMA gathers rows by index [29]. These kernels use the indirection to avoid a copy, but do not characterize its cost against the dense kernel, nor face the overlapping receptive fields that scattering convolution outputs disrupts. We show that for convolution on a dense input grid the indirection leaves the reduction identical to dense, so the residual gap is a per-layer fixed cost rather than lost throughput (§3).

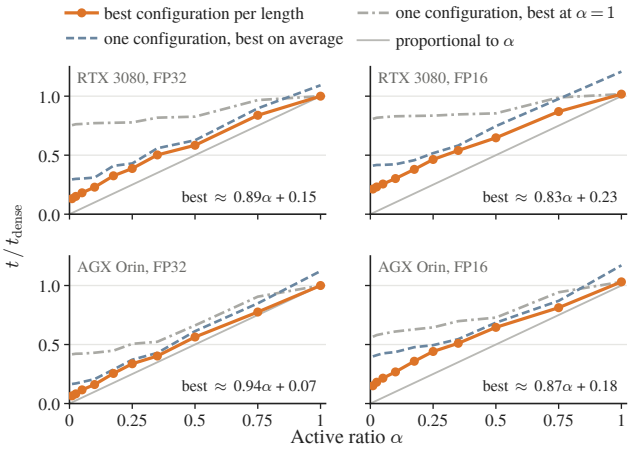


Figure 6. Convolution time over dense time, averaged over the layers of YOLOv8m, against the active ratio. A configuration is a tile shape with its split-K. The per-length best takes the fastest configuration at each worklist length; the two other curves keep one configuration for every length, chosen for the lowest expected time over a uniform α or for the lowest time at $\alpha = 1$.

3 Characterizing Worklist-Driven Convolution

Before presenting WISEConv, we characterize what worklist-driven tiled convolution achieves and where it falls short. We study the compute stage (a dense tiled implicit-GEMM kernel redirected to compute only positions from a worklist; §4.4) and the construction stage (a mask-to-worklist pass) separately, so that each finding belongs to the execution model, not to a particular implementation. FP16 uses `mma.sync` tensor cores; FP32 is a SIMT kernel of the same structure. Each configuration is timed in isolation inside CUDA graphs on RTX 3080 (68 SMs) and Jetson AGX Orin (16 SMs) at locked clocks, using YOLOv8m [22] at batch 1 on WildTrack [5] unless noted otherwise. Dense reference: the faster of the same kernel on the full grid and cuDNN non-Winograd.

3.1 Affine Latency Requires Following the Length

A worklist kernel issues only active outputs ($\eta \approx 1$), so proportional speedup means keeping dense throughput. When we grant the kernel its best configuration at every worklist length, its latency averaged across layers is affine in α, well fitted by $t/t_{\text{dense}} = a\alpha + b$. On RTX 3080, $a = 0.89$, $b = 0.15$ (FP32) and $a = 0.83$, $b = 0.23$ (FP16); on AGX Orin, $a = 0.94$, $b = 0.07$ (FP32) and $a = 0.87$, $b = 0.18$ (FP16), with per-layer R^2 medians of 0.985–0.995 (Figure 6). The slope a matches the dense marginal cost; the entire departure from proportional is the intercept b, a per-layer fixed cost analyzed in §3.4.

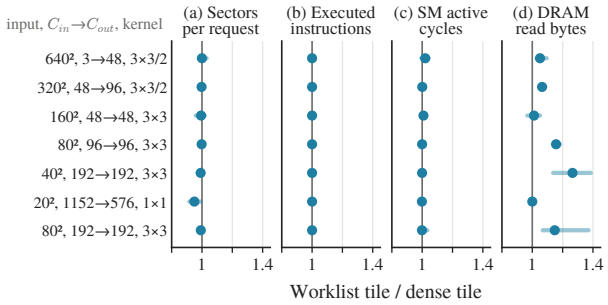


Figure 7. A worklist tile against a dense tile of the same configuration on representative layers (Nsight Compute, cold L2; bar: range over the two GPUs and two precisions, dot: median). (a) Global-load sectors per request of the median frame. (b)–(d) Cost of one more tile: the slope over the tile count of six worklist prefixes over that of six dense prefixes, so per-launch costs such as the cold weight reads cancel.

This affine behavior holds only when the configuration follows the worklist length. At $\alpha \leq 0.05$, the configuration fastest at $\alpha = 1$ runs 6.8× (FP32) and 4.4× (FP16) slower than the per-length best on RTX 3080, and 6.2× and 3.6× on AGX Orin (Figure 6): large tiles amortize weight loads over many rows, but when rows are few, smaller tiles and split-K shorten the chain each SM executes. The best single configuration for a uniform α narrows the gap to 2.0–2.6× at $\alpha \leq 0.05$, at the price of running 9.2–18.7% slower than dense at $\alpha = 1$, and remains 11.2–16.5% slower than the per-length best in expected time.

Insight 1. *The latency of worklist-driven tiled convolution is affine in the active ratio at dense marginal cost only when the tile configuration follows the worklist length.*

3.2 Why Per-Tile Throughput Matches Dense

The worklist changes only the base address of each row; under a channels-last layout, the reduction loop over k^2 taps and channel slices keeps the dense kernel's addressing, so channel coalescing and weight-tile reuse are untouched. Counters confirm it (Figure 7). We compare the cost of one more tile, the slope over the tile count, because a single launch also carries fixed costs (cold weight reads, first misses) that a dense launch spreads over hundreds of tiles and a worklist launch over a few; those costs form the intercept of §3.4. Per added tile, a worklist tile executes the same instructions as a dense tile on every layer, GPU, and precision, and spends 0.98–1.04× its SM cycles; sectors per request stay within 0.92–1.03× of dense. It reads more DRAM bytes, up to 1.39× (median 1.07×), because scattered positions share fewer receptive fields with their neighbors; the 1×1 layer, whose receptive fields do not overlap, reads exactly as much as dense. The extra bytes do not appear in the cycles.

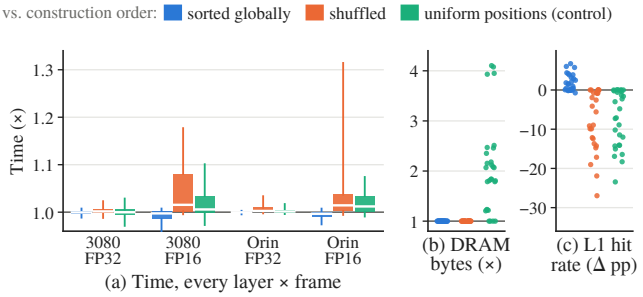


Figure 8. The worklist in other orders, relative to the construction order (same length and tile). Uniform positions are a control: as many positions drawn uniformly over the grid, without the spatial clustering of the mask. (a) Every layer and captured frame (box: 25th–75th percentile, whiskers: 5th–95th, line: median). (b), (c) Representative layers, one dot per layer, GPU, and precision (Nsight Compute, cold L2).

Insight 2. *Irregularity is confined to row-level indirection outside the reduction loop, leaving channel-contiguous access and weight-tile reuse identical to dense execution.*

3.3 Order Is a Second-Order Effect

Scattered worklist entries could hurt L1 reuse by placing positions with non-overlapping receptive fields into the same tile. We compare the construction order (tile-local: dense traversal order within each construction tile, arrival order across tiles) with the same worklist sorted globally in dense traversal order and randomly shuffled. Across about 7,900 layer–frame samples per GPU and precision, global sorting runs at 0.997–1.000× the construction order (median) and saves at most 1.1% of model time; shuffling runs at 1.002–1.015× (Figure 8a). All three orders read the same DRAM bytes (within 0.1%); shuffling only lowers the L1 hit rate, by up to 27 points (Figure 8b, c). This costs little on RTX 3080 FP32 but up to 1.31× (95th percentile) on AGX Orin FP16, where the low-intensity early layers expose the extra L1-to-L2 traffic to Orin's lower L2 bandwidth. Spatial clustering, not order, sets DRAM traffic: the same number of positions drawn uniformly reads up to 4.1× the bytes.

Insight 3. *Worklist order leaves DRAM traffic unchanged; a tile-local order is within 1.1% of a global order.*

3.4 The Residual Gap Is a Per-Layer Fixed Cost

The intercept b is the time of a launch that computes almost nothing. A launch with a single tile takes 69–80% of b (Figure 10a). Without work, a launch spends 65–81% of its stall cycles on constant-cache misses (the kernel parameters are cold); one tile adds barriers and on-chip latency, while DRAM/L2 latency stays at 6–23% (median; Figure 10a). The intercept is 10.0–17.4% of a median layer's dense time, except

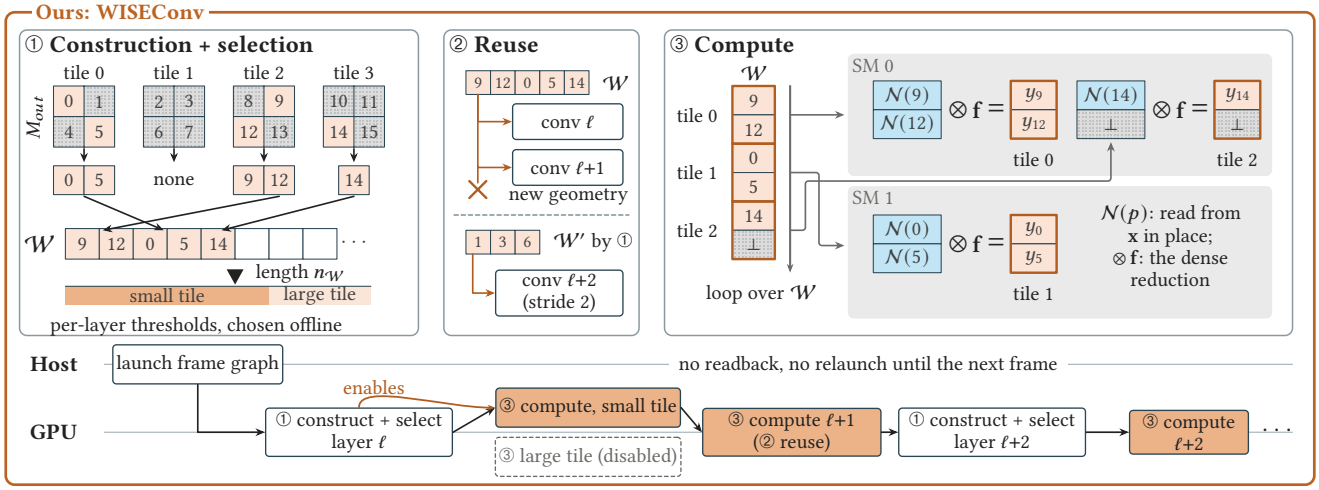


Figure 9. WISEConv on the running example of Figure 2. ① One pass over the output mask M_{out} compacts each construction tile into a segment of its active positions and appends the segments to the worklist $\mathcal{W}$ in arrival order; the same pass compares the worklist length $n_{\mathcal{W}}$ with the layer's offline thresholds and selects the tile configuration. ② Later operators with the same geometry reuse $\mathcal{W}$; an operator that changes the geometry (here a stride-2 convolution) gets a new worklist from ①. ③ A persistent grid loops over $\mathcal{W}$ in tiles of the selected size; each tile reads its receptive fields from **x** in place and runs the dense reduction, so only the padding of the last tile is wasted. Bottom: the host launches the per-frame CUDA graph once, and construction enables one of the layer's candidate compute nodes on the device, so nothing is read back to the host.

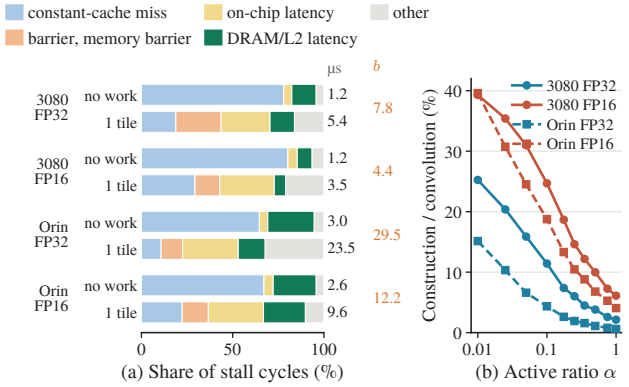


Figure 10. The two per-layer fixed costs. (a) Warp stall cycles of one compute launch with no work and with a single tile, grouped by cause (median over representative layers, Nsight Compute; on-chip latency is short scoreboard and fixed-latency waits), the time per launch, and the intercept b of the per-layer affine fit on the same layers. (b) Model-wide construction time over convolution time with the per-length best configuration, against the active ratio.

5.6% on AGX Orin FP32, where each layer is longer relative to the launch.

Construction adds a second fixed cost. Each construction pass takes 1.6–4.4 μs on RTX 3080 and 3.8–27.2 μs on AGX Orin, almost independent of activity, while the convolution it feeds shrinks with α. Its share of model time therefore rises as activity falls (Figure 10b). On RTX 3080 FP32 the share grows from 3.8% at $\alpha = 0.5$ to 11.4% at $\alpha = 0.1$; on the faster FP16 path it reaches 39.3% at $\alpha = 0.01$.

Insight 4. *The residual gap comes from per-layer fixed costs independent of activity: launch latency in the compute intercept, and near-constant construction time.*

3.5 Following the Length Is Not Free

Insight 1 requires the configuration to follow the worklist length, but the length exists only on the device and changes at every layer of every frame. Every mechanism that selects a kernel from it pays per decision point. Measured in isolation, reading the length back to the host costs 7.5 μs per layer on RTX 3080 and 20.8 μs on AGX Orin, and an IF conditional graph node 9.5 and 32.4 μs; a device-updatable kernel node costs 0.8 μs per layer plus 0.8 μs per candidate switch (1.7 plus 2.0 μs on AGX Orin). The median per-layer gain of following the length is 1.3–2.7 μs on RTX 3080 and 5.0–12.3 μs on AGX Orin (Figure 11). Readback and conditional nodes exceed the gain on 89–100% of layers, the updatable node on 7–25% without switching and on up to 71% when it switches every frame. The mechanism, not the oracle, bounds how many candidates and selection points a frame can afford.

Insight 5. *Every mechanism for following the worklist length incurs a per-decision-point cost often comparable to the per-layer gain from adaptation.*

Insights 4 and 5 together mean that current hardware cannot reach the oracle affine line without trading selection

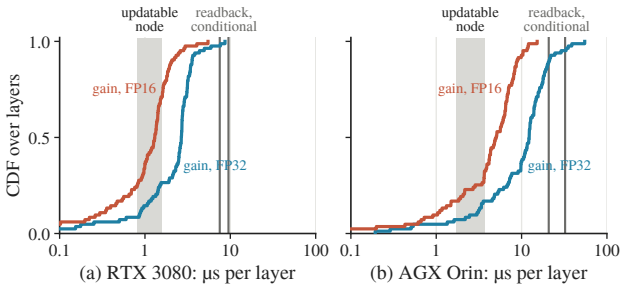


Figure 11. Per-layer gain of following the worklist length (per-length best over the best single configuration, real trace; CDF over the layers of YOLOv8m) against the per-layer cost of each selection mechanism on the same GPU. The band spans the updatable node from no switch to a switch every frame.

cost against adaptation gain; §6.5 discusses the hardware support that would remove this trade-off.

4 WISEConv Design

4.1 Overview

WISEConv exploits the fact that dense tiled convolution is driven by output coordinates: each coordinate determines the receptive-field reads and output write, while the reduction over channels and kernel offsets remains regular. Replacing the dense coordinate sequence with an active-output worklist introduces position-level selectivity into the tiled pipeline without altering the reduction datapath (Figure 9).

Translating this into end-to-end latency reduction requires co-designing two stages. The construction stage (§4.2) derives the worklist from the mask, and compatible operators reuse it to avoid repeated construction (§4.3). The compute stage (§4.4) consumes the worklist through a persistent grid with the dense reduction datapath; because the worklist length varies per frame and is visible only on the device, a cost-aware mechanism selects the tile configuration without host involvement (§4.5).

4.2 Tile-Local Worklist Construction

Constructing a tight schedule already requires a scan of the output grid, but the resulting coordinates must also retain enough spatial structure for efficient consumption. Globally stable compaction preserves the complete dense traversal order but requires a prefix scan or another pass to obtain device-wide offsets; per-coordinate appends avoid that coordination but scatter neighboring positions. Materializing active receptive fields, as gather-scatter does, regularizes the compute input at the cost of feature traffic proportional to the active count and $k^2 C_{in}$. The useful middle ground is therefore a metadata-only schedule with local rather than global ordering.

Algorithm 1 Tile-Local Worklist Construction

Require: Input mask M_{in}, output grid $\mathcal{G}$, tile size B_τ
Ensure: Mask M_{out}, worklist $\mathcal{W}$, length $n_\mathcal{W} = |\mathcal{W}|$

1: $n_\mathcal{W} \leftarrow 0$
2: **for** each $B_\tau \times B_\tau$ spatial tile τ **in parallel do**
3: **for** each position $p \in \tau$ **in parallel do**
4: active$(p) \leftarrow \exists\, r \in \mathcal{N}(p) : M_{in}[r] = 1$
5: $M_{out}[p] \leftarrow$ active(p)
6: **end for**
7: $\mathcal{P}_\tau \leftarrow$ Compact(τ, M_{out})
8: $n_\tau \leftarrow |\mathcal{P}_\tau|$
9: base $\leftarrow$ AtomicAdd$(n_\mathcal{W}, n_\tau)$
10: **for** each $0 \le j < n_\tau$ **in parallel do**
11: $\mathcal{W}[\text{base} + j] \leftarrow \mathcal{P}_\tau[j]$
12: **end for**
13: **end for**

WISEConv realizes this with contiguous, tile-local worklist segments. Because the number of active positions is not known before the pass, it preallocates a coordinate buffer with capacity $|\mathcal{G}| = H_{out} W_{out}$, the static upper bound on $|\mathcal{W}|$. Algorithm 1 details the resulting single pass. Line 1 initializes the device-resident length counter. Lines 2–7 partition the output grid into $B_\tau \times B_\tau$ construction tiles, derive the corresponding entries of M_{out} using Eq. 1, and compact each tile's active coordinates into a local sequence $\mathcal{P}_\tau$ in dense-traversal order. When a learned gate supplies M_{out} directly [24, 40], Lines 4–5 are omitted and Line 7 compacts the supplied mask. Lines 8–11 count the compacted coordinates, atomically reserve [base, base + n_τ) from the counter, and copy $\mathcal{P}_\tau$ into that interval. After all tiles complete, the reserved intervals collectively store $\mathcal{W}$, and the counter records $n_\mathcal{W}$. Because the layer shape fixes the buffer capacity and launch geometry while only the device observes the data-dependent length, construction requires no runtime allocation or host-visible active count.

To relate the worklist to the useful-compute ratio of Eq. 3, we insert its length $n_\mathcal{W}$ between the required-position count and the issued-slot count. For $n_\mathcal{W} > 0$,

$$\eta = \underbrace{\frac{\|M_{out}\|_0}{n_\mathcal{W}}}_{\eta_{\text{construct}}} \cdot \underbrace{\frac{n_\mathcal{W}}{|C|}}_{\eta_{\text{compute}}} . \tag{5}$$

We analyze $\eta_{\text{construct}}$ (required positions vs. worklist entries) here and η_{compute} (worklist entries vs. issued slots) in §4.4; the factorization decomposes η only, with all stage costs reflected in throughput T.

Because construction tiles partition the output grid and use disjoint reservations, each active coordinate appears exactly once, so $n_\mathcal{W} = \|M_{out}\|_0$. Fresh construction therefore has $\eta_{\text{construct}} = 1$ for a nonempty worklist.

WISEConv thus confines its ordering guarantee to each contiguous segment: coordinates retain the tile's dense traversal, while independently reserved segments provide no cross-boundary locality guarantee. A compute tile of B_M positions (the output-position tile width used by the compute stage, §4.4) may therefore span coordinates from multiple construction segments when segments are shorter than B_M; the preserved locality is intra-segment contiguity, not compute-tile-to-construction-tile alignment. As shown in §6.4, this tile-local order achieves compute-stage throughput comparable to global ordering, making the additional ordering pass an unfavorable end-to-end trade-off.

4.3 Cross-Operator Worklist Reuse

Each construction pass examines all $H_{out}W_{out}$ positions and, when deriving M_{out} from M_{in}, performs $O(k^2)$ mask work per position. This channel-independent grid scan contrasts with required convolution work, which scales with the active ratio, spatial extent, and both channel dimensions. WISEConv therefore amortizes construction by reusing one worklist across each compatible layer sequence.

For layer l, let $\mathcal{G}^l$ denote its output grid, M^l_{out} its output mask, $\mathcal{W}^l$ its worklist, and $n^l_{\mathcal{W}}$ its device-resident length. WISEConv propagates the mask and worklist metadata alongside the feature tensor. Whether two consecutive layers are reuse-compatible depends on their operator types, not on runtime mask values. An operator that preserves the output grid and does not expand the active set guarantees, for any input mask, that $\mathcal{G}^{l+1} = \mathcal{G}^l$ and

$$\{p \in \mathcal{G}^{l+1} \mid M^{l+1}_{out}[p] = 1\} \subseteq \{p \in \mathcal{G}^l \mid M^l_{out}[p] = 1\}. \quad (6)$$

WISEConv checks this condition once at model-conversion time. Because $\mathcal{W}^l$ covers M^l_{out}, Eq. 6 guarantees coverage for M^{l+1}_{out}. WISEConv therefore forwards $\mathcal{W}^{l+1} = \mathcal{W}^l$ and $n^{l+1}_{\mathcal{W}} = n^l_{\mathcal{W}}$ without reconstruction and applies the condition recursively at each subsequent layer.

Equality in Eq. 6 covers same-grid 1×1 convolutions, mask-preserving activations, and inference-time normalization, leaving $\eta_{\text{construct}}$ unchanged. Strict inclusion can result from mask-narrowing operators in the upstream policy, such as the activation-fused update truncation that DeltaCNN [32] uses to increase downstream sparsity. The inherited worklist remains a valid superset, trading a lower $\eta_{\text{construct}}$ for one fewer construction pass without violating coverage. Because compute gates all output writes by the exact mask, positions scheduled but no longer active under the current mask participate in reduction yet produce no store; the superset schedule therefore cannot alter any inactive position's observable state.

A change in the output grid violates the geometry constraint, while receptive-field expansion may violate Eq. 6 by introducing positions outside the upstream mask; either case invokes the construction pass of §4.2 to emit a new tight worklist. A lower $\eta_{\text{construct}}$ alone does not trigger reconstruction because tightening a valid superset would pay another grid scan. Common receptive-field-expanding layers such as 3×3 convolutions still create rebuild points throughout the network, leaving device-resident worklist lengths variable across layers and frames. Compute must therefore handle a dynamic problem size.

4.4 Worklist-Driven Tiled Compute

4.5 Cost-Aware Tile Selection

5 Implementation

We implement WISEConv as a PyTorch extension with a Python frontend and a CUDA backend written in C++. A model-conversion pass replaces supported modules, preserves their parameters and weights, and statically marks worklist-reuse edges and rebuild points. The compute backend adapts CUTLASS [1] dense tiled implicit-GEMM pipelines for direct worklist consumption, covering both FP32 (SIMT) and FP16 (tensor core) arithmetic with NHWC tensors; the full-workload comparison uses the FP32 datapath, while sensitivity studies additionally exercise the FP16 datapath. The backend covers all convolution variants used by the evaluated models, as well as mask-aware auxiliary operators. The partial-reduction workspace $\mathbf{Z}$ is shared across all layers at the maximum required size and allocated once during initialization. Every backend variant agrees numerically with cuDNN at each active output position.

6 Evaluation

6.1 Experimental Setup

We ask three questions: whether WISEConv turns spatial sparsity into end-to-end latency and energy reduction across GPU classes (§6.2), where its remaining time goes (§6.3), and whether each co-designed component is necessary (§6.4).

Platforms. We evaluate six NVIDIA GPUs that span datacenter, desktop, laptop, and embedded classes: A100, RTX 4090, RTX 3080, RTX 4070 Laptop, Jetson AGX Orin, and Jetson Xavier NX. GPU clocks are locked on every platform so that latencies are stable across runs. On the two Jetson boards, where the energy budget constrains deployment, we also report per-frame energy from the board's input power.

Workloads. The three workloads (Table 1) cover the three mask sources of §2.1: event masks, temporal-difference masks, and learned spatial gates. Each mask policy follows its upstream method; where it needs thresholds, they are tuned on data disjoint from the evaluation set.

Baselines. *Dense* runs the original model with PyTorch and cuDNN [7] autotuning: it computes every position and carries none of the masking machinery (no mask generation, propagation, or temporal state). *Tile skipping* uses DeltaCNN's kernels [32], and *gather-scatter* uses MMCV's

Table 1. Workloads.

	FireFlowNet [30]	**YOLOv8m** [22]	**DynConv** [40]
Task	Optical flow	Detection	Human pose
Mask source	Ev-Conv [10]	DeltaCNN [32]	Learned gates [40]
Dataset	MVSEC [48]	WildTrack [5]	MPII [2]
Input	256×256	640×640	256×256
Parameters	0.056M	25.9M	6.89M
Eval. frames	196.8K	251.7K	2,958

masked convolution [6] (DynConv's own CUDA backend [40] for pose). All paths run the same weights and inputs, and the three masked paths the same layer masks, so they differ only in the masked-convolution backend. Every path except gather-scatter replays as a CUDA graph; gather-scatter reads each layer's active count on the host to size its buffers, so it must run eagerly. The baselines implement only FP32, so FP16 compares WISEConv with Dense alone. We report WISEConv's speedup over the fastest baseline on each workload and platform.

Protocol. Latency is the GPU time of one frame measured with CUDA events; it includes worklist construction and all non-convolution operators. YOLOv8m processes whole videos continuously, so its temporal masks evolve as in deployment, at batch 1 (YOLOv8m b1: a single latency-bound camera, our primary setting) and batch 4 (YOLOv8m b4: four concurrent camera streams). WISEConv's per-device tuning (§4.5) runs once offline and is excluded. Task accuracy is reported to confirm that the sparse paths agree.

6.2 End-to-End Performance

Latency. WISEConv is the fastest path in all 24 measured workload-platform pairs (Figure 12). Relative to the fastest evaluated competing path in each pair, it achieves a 1.58× geometric-mean speedup; individual-pair speedups range from 1.33× to 2.12×, corresponding to 25.0–52.8% lower latency. Gains span more than two orders of magnitude in absolute latency, from 0.23 ms for FireFlowNet on RTX 4090 to 88.6 ms for YOLOv8m on Xavier NX.

The two existing sparse paths split the remaining 24 pairs between them: dense is fastest in 13 and tile skipping in 11; gather-scatter never wins. The split tracks active ratio: at the YOLO workloads' $\alpha \approx 48$–49%, few tiles are fully inactive, so tile skipping cannot compensate for its lower throughput; on FireFlowNet ($\alpha \approx 25\%$) and DynConv ($\alpha \approx 16\%$), enough tiles are skippable for it to overtake dense. Neither path is uniformly strong, and WISEConv outperforms whichever wins. On the Jetson platforms the lead narrows (geometric mean 1.53× vs. 1.61× on discrete GPUs) but WISEConv remains fastest in all eight pairs.

Across all 24 pairs, WISEConv averages 1.76× the ideal proportional-scaling reference (dashed lines in Figure 12), compared with 2.79× for the per-pair fastest competing path.

Table 2. Board energy per frame (mJ) on the Jetson platforms. Parentheses give each masked path's change relative to Dense of the same precision; WISEConv is in bold.

		FireFlowNet	YOLOv8m b1	YOLOv8m b4	DynConv
AGX Orin	FP32 Dense	165	2102	2212	951
	FP32 Tile skipping	79.9 (−51.5%)	752 (−64.2%)	572 (−74.1%)	467 (−50.9%)
	FP32 Gather-scatter	205 (+24.2%)	791 (−62.4%)	543 (−75.5%)	2324 (+144.3%)
	FP32 WISEConv	**48.6** (−70.5%)	**363** (−82.7%)	**280** (−87.3%)	**258** (−72.8%)
	FP16 Dense	47.7	432	375	314
	FP16 WISEConv	**17.5** (−63.3%)	**101** (−76.5%)	**61.6** (−83.6%)	**123** (−60.8%)
Xavier NX	FP32 Dense	248	2505	2425	1067
	FP32 Tile skipping	114 (−54.0%)	876 (−65.0%)	837 (−65.5%)	499 (−53.3%)
	FP32 Gather-scatter	299 (+20.2%)	1028 (−59.0%)	818 (−66.3%)	2519 (+136.1%)
	FP32 WISEConv	**76.8** (−69.1%)	**539** (−78.5%)	**450** (−81.4%)	**332** (−68.9%)

Table 3. Task accuracy on RTX 3080 (mAP over IoU 0.5–0.95). Dense runs the unmasked model; the three masked paths consume the same layer masks.

		FireFlowNet	YOLOv8m b1		DynConv
		AEE ↓	AP50 ↑	mAP ↑	PCKh ↑
FP32	Dense	1.903	0.632	0.205	88.06
	Tile skipping	1.888	0.639	0.209	86.68
	Gather-scatter	1.888	0.639	0.209	86.68
	WISEConv	1.888	0.640	0.210	86.68
FP16	Dense	1.903	0.633	0.205	88.06
	WISEConv	1.889	0.639	0.210	86.66

The gap to ideal tracks active ratio: on the YOLO workloads WISEConv runs at 1.36× ideal, while on DynConv ($\alpha \approx 16\%$) construction and fixed costs widen it to 2.60×.

Energy. WISEConv is the lowest-energy path for every workload on both Jetson platforms (Table 2). In FP32 it uses 32.8–51.8% less energy per frame than the lowest-energy baseline, and in FP16 60.8–83.6% less than Dense.

Accuracy. WISEConv's latency and energy gains do not change accuracy (Table 3). Given the same layer masks, the three masked paths agree: AEE and PCKh are identical, and YOLOv8m AP differs by at most 0.1 points, a residual of floating-point reduction order rather than of covered positions. Where worklist reuse computes a superset of the required positions, the output mask gates writeback, so the extra positions never reach downstream layers. The remaining gap to Dense (1.4 PCKh points lower on DynConv, slightly better AEE and AP on the other two) is set by the upstream mask policy, which WISEConv consumes without modification.

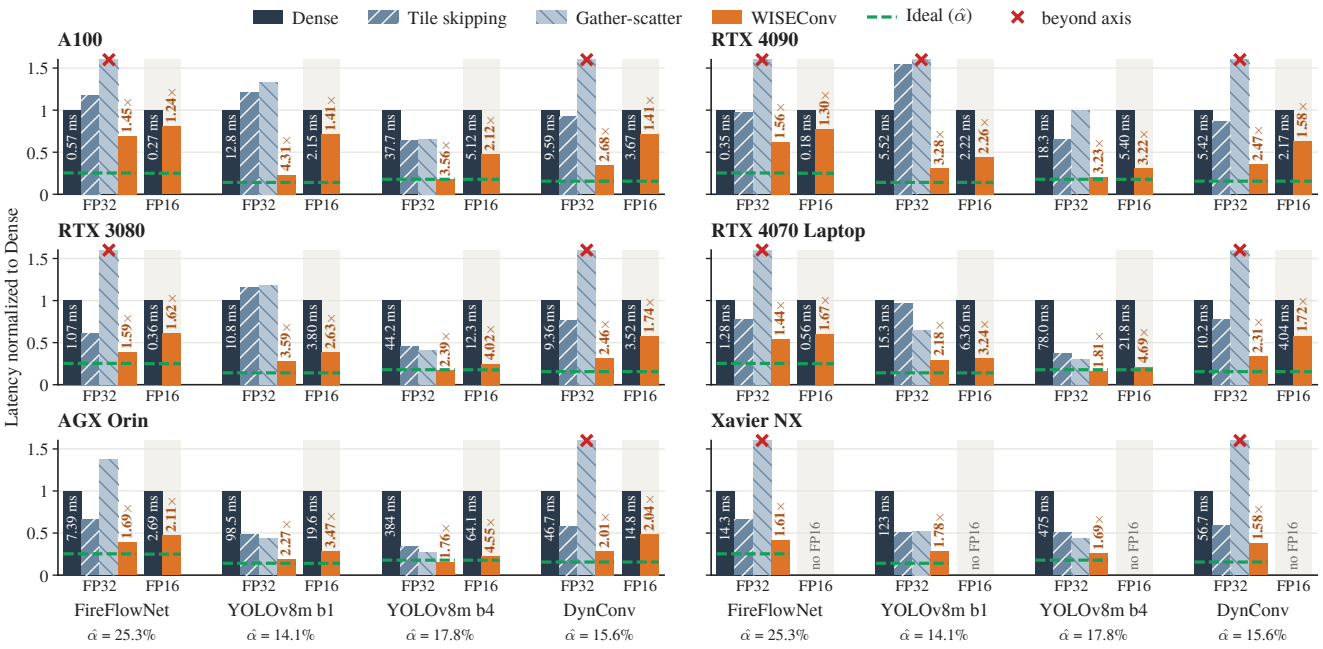


Figure 12. Full-model latency on six GPUs, normalized to Dense of the same precision (lower is better). Each workload has an FP32 group with all four paths and an FP16 group with Dense and WISEConv, since the baselines have no FP16 path. Dense bars give the absolute latency (per frame; per batch of four for YOLOv8m b4), WISEConv bars give its speedup over the fastest other path of the group, and a red cross marks bars that extend beyond the axis. The dashed line is ideal proportional scaling, the mean active ratio $\hat{\alpha}$ given under each workload.

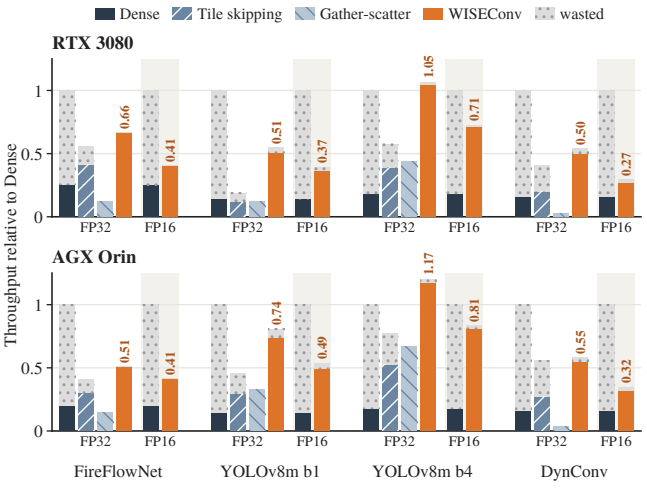


Figure 13. Effective throughput ηT (colored) and wasted throughput $(1-\eta)T$ (dotted) of every path, relative to the raw throughput of Dense of the same precision. WISEConv bars give its effective throughput.

6.3 Microbenchmarks

Effective throughput. A masked path lowers latency only if it is both selective, captured by the useful-compute ratio η, and fast, captured by raw throughput T (Eq. 4). Figure 13 reports both, and the two baselines fail on opposite axes. Tile skipping keeps the dense tile schedule but computes every position of an active tile, so its η is only 0.49–0.74 while its raw throughput reaches 19.0–77.5% of Dense. Gather-scatter issues exactly the required positions ($\eta = 1$), but extraction and writeback hold its raw throughput to 2.6–66.5% of Dense. WISEConv keeps η at 0.91–1.00 and sustains 50.8–120.2% of Dense raw throughput in FP32 (29.6–83.6% in FP16), the highest of the masked paths in every case. On YOLOv8m b4 its effective throughput exceeds even Dense's raw throughput, because cuDNN runs this configuration slower than WISEConv's kernels.

Table 4. WISEConv η decomposition and worklist-length statistics on RTX 3080 (FP32; batch 1 unless noted). The $n_{\mathcal{W}}$ percentiles are computed from worklist lengths pooled across all layers and frames.

Workload	$\eta_{\text{construct}}$	η_{compute}	η	$n_{\mathcal{W}}$		
				P10	P50	P90
FireFlowNet	100.0%	99.8%	99.8%	529	13,651	36,009
YOLOv8m	100.0%	92.4%	92.4%	24	261	1,481
YOLOv8m b4	100.0%	98.1%	98.1%	184	1,410	7,256
DynConv	100.0%	92.1%	92.1%	3	43	389

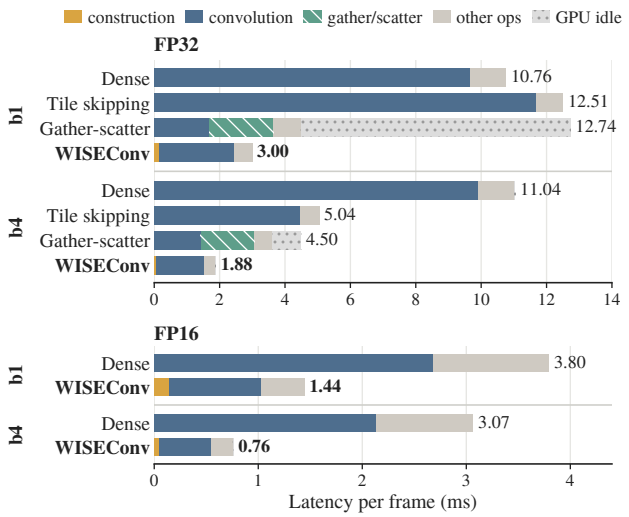


Figure 14. YOLOv8m latency per frame by stage on RTX 3080, at batch 1 (b1) and 4 (b4), with FP32 and FP16 on separate axes. Stages are each path's kernel shares of GPU busy time, scaled to its end-to-end latency; GPU idle is the remaining time in the frame. Gather-scatter's gather/scatter includes compacting the mask into the index list it gathers by.

Table 4 localizes WISEConv's own loss through the decomposition of Eq. 5. $\eta_{\text{construct}}$ is 100.0% on every workload: tile-local compaction emits each active coordinate exactly once. All of the loss is in η_{compute}, the fraction of issued slots that are worklist entries, and it tracks worklist length. FireFlowNet and YOLOv8m b4, whose median worklists hold thousands of entries, reach 99.8% and 98.1%; YOLOv8m b1 and DynConv, whose worklists are often shorter than a few tiles (P10 of 24 and 3 entries), lose 7.6% and 7.9% to the padding of their final tiles.

Latency decomposition. Figure 14 splits YOLOv8m latency by stage. At batch 1 in FP32, WISEConv cuts convolution from 9.68 to 2.27 ms (4.3×) at the cost of 0.17 ms of construction, and the frame falls from 10.76 to 3.00 ms. Gather-scatter issues the least convolution of any masked path (1.68 ms), but its per-layer host synchronization leaves the GPU idle for 8.26 ms, 64.8% of its frame. Batching four streams hides most of that idle time (0.87 ms per frame) and lets both baselines fill the GPU, so tile skipping and gather-scatter fall to 5.04 and 4.50 ms per frame; WISEConv still leads at 1.88 ms because its batch-1 grid already keeps the GPU busy. In FP16, tensor cores shrink convolution more than the rest of the frame: construction and other operators grow from 24.3% of WISEConv's FP32 frame to 39.6%, and the speedup over Dense narrows from 3.59× to 2.63×.

Memory overhead. Table 5 reports peak live GPU allocation. FireFlowNet, YOLOv8n, and YOLOv8m use temporal mask policies that retain previous-frame activations for differencing, so every sparse path allocates 3–4× Dense. The temporal state dominates the gap, and WISEConv stays within the same tier as tile skipping; gather-scatter is the exception on YOLOv8m, where materializing input patches for each active position adds to the temporal state and pushes it above both other sparse paths. DynConv applies learned spatial gates without temporal state, placing all backends in a narrow 27–34 MiB range. Here WISEConv's worklist and coordinate metadata add only 2.3 MiB over Dense, while gather-scatter is the most compact because DynConv's predominantly pointwise block structure lets intermediates stay in the gathered compact representation [40].

Table 5. Peak live GPU allocation (MiB) on RTX 3080 (FP32; batch 1 unless noted).

System	FireFlowNet	YOLOv8m	YOLOv8m b4	DynConv
Dense	33.9	182.0	432.2	56.8
Tile skipping	141.8	751.0	2701.0	53.8
Gather-scatter	197.0	834.8	2737.0	57.2
WISEConv	128.6	763.9	2730.4	56.9

6.4 Ablation Study

Table 6 isolates each co-designed component by disabling it while holding all others fixed on YOLOv8n; the stage decomposition separates the effect on construction and convolution.

Construction. On RTX 3080, Atomic append and Global order raise construction from 0.12 to 0.43 and 0.54 ms without reducing convolution (0.94 ms throughout), adding 19.1% and 19.6% to the end-to-end latency over Ours; Global order's higher construction reflects its additional kernel passes. No reuse *raises* η to 99.7% ($\eta_{\text{construct}} = 1$ per layer) but increases construction 1.6× (0.12 to 0.19 ms), adding 5.7% over Ours: rebuilding the worklist every layer is more expensive than reusing a slightly loose superset worklist. On AGX Orin the same pattern holds with smaller margins: construction variants add 1–9% over Ours while convolution stays within 0.05 ms, because convolution is a larger share of the total.

Compute. On RTX 3080, DP only raises convolution from 0.94 to 1.01 ms over Ours (+0.07 ms); No hybrid, which retains full Stream-K but not the hybrid mode, recovers most of that gap (0.96 ms, +0.02 ms over Ours). Readback & launch runs the same kernels as Ours but adds per-operator host synchronization, inflating the total from 1.66 to 7.61 ms (4.58× Ours). On AGX Orin, DP only adds 1.5% end-to-end latency over Ours and readback inflates it to 2.38× Ours.

6.5 Discussion

Limitations. WISEConv converts spatial sparsity into latency, so its benefit is bounded by how sparse the upstream

Table 6. Co-design ablation on YOLOv8m (batch 1, RTX 3080, FP16). Each variant replaces one mechanism and keeps the weights, inputs, upstream masks and every other component. Cons. and Conv. are the construction- and compute-stage GPU kernel time (ms); Tot. is the full-model latency (ms, aligned to the whole-video end-to-end latency of Ours); × is relative to Ours unless noted.

	Variant	Cons.	Conv.	Tot.	×
Cons.	No reuse	0.30	0.86	1.59	1.10
Comp.	Shuffled order	0.15	0.92	—	1.04†
	Readback & launch	—	—	9.40	6.51
	Full-layer grid	0.15	1.06	1.63	1.13†
Sel.	Dense-tuned tile	0.16	1.89	2.47	1.71
	Best single tile	0.16	0.88	1.47	1.02
	Latency-optimal tile	0.16	1.00	1.59	1.10
	Cost-unaware $K_{\max}$	0.22	0.85	1.55	1.07
	Zero-cost oracle	—	—	1.32	0.92
	Ours	0.16	0.86	1.44	1.00

Variants. No reuse: rebuild the worklist at every layer. *Shuffled order*: permute each worklist on the device before the compute stage (permutation not counted; convolution time). *Readback & launch*: read n_W back to the host per layer and launch the exact grid. *Full-layer grid*: launch the dense layer's grid; CTAs beyond the worklist exit. *Dense-tuned / Best single / Latency-optimal tile*: Ours' candidate structure (the same layers, candidate counts and thresholds), every candidate the tile fastest at α=1, with the best expected speedup, or with the lowest expected latency (uniform n_W). *Cost-unaware $K_{\max}$*: up to four candidates at every layer, ignoring their cost. *Zero-cost oracle*: those candidates, each frame on a graph captured for its own n_W, with no selection mechanism. †Relative to the best single tile without the selection mechanism.

masks are and by how much of the frame convolution occupies. First, the gain vanishes as the active ratio approaches one: at full activity the per-length best configuration only matches dense execution (§3.1) while construction still runs, so a fully active input runs slightly slower than the dense path. WISEConv consumes masks and makes no claim about their quality or the resulting accuracy; a policy that keeps most positions active should stay on the dense path. Second, WISEConv accelerates convolution only, so convolution's share of frame latency caps the end-to-end speedup. FP16 tensor cores shrink that share, and the remaining non-convolution operators become the next bottleneck, which is why FP16 speedups are narrower than FP32 ones (§6.3). Models dominated by other operators bound the gain further.

Hardware implications. The characterization traces the remaining gap to two per-layer costs that current GPUs leave to software. The intercept of the affine model is launch and dependency latency rather than lost bandwidth (Insight 4), and its share grows as inputs become sparser; hardware that shortens the path from launch to the first useful instruction (for example, by staging kernel parameters before the CTA starts) and the dependent chain of a single tile would move sparse execution toward the proportional line. Following the worklist length costs about as much per decision point as it gains (Insight 5), because every existing mechanism either returns to the host or reconfigures the graph. A launch predicate that the hardware evaluates on a device-resident value before allocating CTA resources, or a work distributor that picks the tile shape from the worklist length, would let every layer adapt its configuration at negligible cost.

7 Conclusion

We presented WISEConv, a masked-convolution runtime that replaces the dense tiled pipeline's contiguous coordinate stream with an active-output worklist, computing almost exclusively the positions an upstream mask requires while retaining the dense tensor layout and the regular per-position reduction. The construction and consumption stages are co-designed to secure both useful-compute ratio and throughput; our ablation confirms that disabling any single mechanism degrades one axis without improving the other. The resulting higher effective throughput lets WISEConv convert upstream mask sparsity into proportional latency and energy reduction for the latency-critical vision tasks where masked convolution applies.

References

[1] 2026. CUTLASS: CUDA Templates and Python DSLs for High-Performance Linear Algebra. https://github.com/NVIDIA/cutlass

[2] Mykhaylo Andriluka, Leonid Pishchulin, Peter Gehler, and Bernt Schiele. 2014. 2D Human Pose Estimation: New Benchmark and State of the Art Analysis. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3686–3693.

[3] Rik J. Bouwmeester, Federico Paredes-Vallés, and Guido C. H. E. de Croon. 2023. NanoFlowNet: Real-time Dense Optical Flow on a Nano Quadcopter. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 1996–2003.

[4] Lukas Cavigelli, Philippe Degen, and Luca Benini. 2017. CBinfer: Change-Based Inference for Convolutional Neural Networks on Video Data. In *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*. 1–8.

[5] Tatjana Chavdarova, Pierre Baqué, Stéphane Bouquet, Andrii Maksai, Cijo Jose, Timur M. Bagautdinov, Louis Lettry, Pascal Fua, Luc Van Gool, and François Fleuret. 2018. WILDTRACK: A Multi-Camera HD Dataset for Dense Unscripted Pedestrian Detection. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 5030–5039. doi:10.1109/CVPR.2018.00528

[6] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. 2019. MMDetection: Open MMLab Detection Toolbox and Benchmark. *CoRR* abs/1906.07155 (2019).

[7] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR* abs/1410.0759 (2014).

[8] Christopher B. Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[9] David B. D'Ambrosio, Saminda Abeyruwan, Laura Graesser, Atil Iscen, Heni Ben Amor, Alex Bewley, Barney J. Reed, Krista Reymann, Leila Takayama, Yuval Tassa, Krzysztof Choromanski, Erwin Coumans,

Deepali Jain, Navdeep Jaitly, Natasha Jaques, Satoshi Kataoka, Yuheng Kuang, Nevena Lazic, Reza Mahjourian, Sherry Moore, Kenneth Oslund, Anish Shankar, Vikas Sindhwani, Vincent Vanhoucke, Grace Vesom, Peng Xu, and Pannag R. Sanketi. 2025. Achieving Human Level Competitive Robot Table Tennis. In *IEEE International Conference on Robotics and Automation, ICRA 2025, Atlanta, GA, USA, May 19-23, 2025*. IEEE, 74–82. doi:10.1109/ICRA55743.2025.11127501

[10] Sankeerth Durvasula, Yushi Guan, and Nandita Vijaykumar. 2023. Ev-Conv: Fast CNN Inference on Event Camera Inputs for High-Speed Robot Perception. *IEEE Robotics Autom. Lett.* 8, 6 (2023), 3174–3181.

[11] Davide Falanga, Philipp Foehn, Peng Lu, and Davide Scaramuzza. 2018. PAMPC: Perception-Aware Model Predictive Control for Quadrotors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 1–8. doi:10.1109/IROS.2018.8593739

[12] Davide Falanga, Suseong Kim, and Davide Scaramuzza. 2019. How Fast Is Too Fast? The Role of Perception Latency in High-Speed Sense and Avoid. *IEEE Robotics Autom. Lett.* 4, 2 (2019), 1884–1891. doi:10.1109/LRA.2019.2898117

[13] Davide Falanga, Kevin Kleber, and Davide Scaramuzza. 2020. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci. Robotics* 5, 40 (2020), 9712. doi:10.1126/SCIROBOTICS.AAZ9712

[14] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 253–267. doi:10.1145/3620666.3651378

[15] Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry P. Vetrov, and Ruslan Salakhutdinov. 2017. Spatially Adaptive Computation Time for Residual Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 1790–1799.

[16] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2023. MegaBlocks: Efficient Sparse Training with Mixture-of-Experts. In *Proceedings of the Sixth Conference on Machine Learning and Systems, MLSys 2023, Miami, FL, USA, June 4-8, 2023*, Dawn Song, Michael Carbin, and Tianqi Chen (Eds.). mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2023/hash/5a54f79333768effe7e8927bcccffe40-Abstract-mlsys2023.html

[17] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

[18] Yizhao Gao, Baoheng Zhang, Xiaojuan Qi, and Hayden Kwok-Hay So. 2023. DPACS: Hardware Accelerated Dynamic Neural Network Pruning through Algorithm-Architecture Co-design. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 237–251. doi:10.1145/3575693.3575728

[19] Benjamin Graham, Martin Engelcke, and Laurens van der Maaten. 2018. 3D Semantic Segmentation with Submanifold Sparse Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[20] Amirhossein Habibian, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. 2021. Skip-Convolutions for Efficient Video Processing. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2695–2704.

[21] Drew Hanover, Antonio Loquercio, Leonard Bauersfeld, Angel Romero, Robert Penicka, Yunlong Song, Giovanni Cioffi, Elia Kaufmann, and Davide Scaramuzza. 2024. Autonomous Drone Racing: A Survey. *IEEE Trans. Robotics* 40 (2024), 3044–3067. doi:10.1109/TRO.2024.3400838

[22] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. 2023. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

[23] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman P. Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (2017).

[24] Fanrong Li, Gang Li, Xiangyu He, and Jian Cheng. 2021. Dynamic Dual Gating Neural Networks. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*. 5310–5319.

[25] Shih-Chieh Lin, Yunqi Zhang, Chang-Hong Hsu, Matt Skach, Md. Enamul Haque, Lingjia Tang, and Jason Mars. 2018. The Architectural Implications of Autonomous Driving: Constraints and Acceleration. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2018, Williamsburg, VA, USA, March 24-28, 2018*, Xipeng Shen, James Tuck, Ricardo Bianchini, and Vivek Sarkar (Eds.). ACM, 751–766. doi:10.1145/3173162.3173191

[26] Renyuan Liu, Yuyang Leng, Shilei Tian, Shaohan Hu, Chun-Fu (Richard) Chen, and Shuochao Yao. 2024. DynaSpa: Exploiting Spatial Sparsity for Efficient Dynamic DNN Inference on Devices. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems, SenSys 2024, Hangzhou, China, November 4-7, 2024*. ACM, 422–435. doi:10.1145/3666025.3699348

[27] Nico Messikommer, Daniel Gehrig, Antonio Loquercio, and Davide Scaramuzza. 2020. Event-Based Asynchronous Sparse Convolutional Networks. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 415–431.

[28] Asit K. Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating Sparse Deep Neural Networks. *CoRR* abs/2104.08378 (2021).

[29] NVIDIA. 2026. Parallel Thread Execution ISA Version 9.5. https://docs.nvidia.com/cuda/parallel-thread-execution/

[30] Federico Paredes-Vallés and Guido C. H. E. de Croon. 2021. Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3445–3454.

[31] Mathias Parger, Chengcheng Tang, Thomas Neff, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2023. MotionDeltaCNN: Sparse CNN Inference of Frame Differences in Moving Camera Videos with Spherical Buffers and Padded Convolutions. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. 17246–17255.

[32] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2022. DeltaCNN: End-to-End CNN Inference of Sparse Frame Differences in Videos. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 12487–12496.

[33] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David R. Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 683–698. doi:10.1145/3620665.3640426

[34] Mengye Ren, Andrei Pokrovsky, Bin Yang, and Raquel Urtasun. 2018. SBNet: Sparse Blocks Network for Fast Inference. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 8711–8720.

[35] Spconv Contributors. 2022. Spconv: Spatially Sparse Convolution Library. https://github.com/traveller59/spconv.

[36] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 13438–13444.

[37] Shawn Tan, Yikang Shen, Rameswar Panda, and Aaron C. Courville. 2024. Scattered Mixture-of-Experts Implementation. *CoRR* abs/2403.08245 (2024). arXiv:2403.08245 doi:10.48550/ARXIV.2403.08245

[38] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *Proceedings of Machine Learning and Systems (MLSys)*.

[39] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 225–239.

[40] Thomas Verelst and Tinne Tuytelaars. 2020. Dynamic Convolutions: Exploiting Spatial Sparsity for Faster Inference. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2320–2329.

[41] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martin Martin, Cewu Lu, Li Fei-Fei, and Silvio Savarese. 2019. DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3343–3352.

[42] Kunyun Wang, Shuo Yang, Jieru Zhao, Wenchao Ding, Quan Chen, Jingwen Leng, and Minyi Guo. 2025. SparseTem: Boosting the Efficiency of CNN-Based Video Encoders by Exploiting Temporal Continuity. In *Advanced Parallel Processing Technologies - 16th International Symposium, APPT 2025, Athens, Greece, July 13-16, 2025, Proceedings*. 246–256.

[43] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. 2019. FastDepth: Fast Monocular Depth Estimation on Embedded Systems. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 6101–6108.

[44] Zhenda Xie, Zheng Zhang, Xizhou Zhu, Gao Huang, and Stephen Lin. 2020. Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 531–548.

[45] Jiacheng Yang, Christina Giannoula, Jun Wu, Mostafa Elhoushi, James Gleeson, and Gennady Pekhimenko. 2024. Minuet: Accelerating 3D Sparse Convolutions on GPUs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. 786–802.

[46] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[47] Changqian Yu, Jingbo Wang, Chao Peng, Changxin Gao, Gang Yu, and Nong Sang. 2018. BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 334–349.

[48] Alex Zihao Zhu, Dinesh Thakur, Tolga Özaslan, Bernd Pfrommer, Vijay Kumar, and Kostas Daniilidis. 2018. The Multi Vehicle Stereo Event Camera Dataset: An Event Camera Dataset for 3D Perception. *CoRR* abs/1801.10202 (2018).